

tf.compat.v1.nn.raw_rnn Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/nn/raw_rnn

Creates an RNN specified by RNNCell cell and loop function loop_fn.

Function Signatures

```
tf.compat.v1.nn.raw_rnn( cell, loop_fn,  
parallel_iterations=None, swap_memory=False, scope=None )
```

Code Examples

```
tf.compat.v1.nn.raw_rnn(  
cell, loop_fn, parallel_iterations=None, swap_memory=False,  
scope=None  
)
```

```
tf.compat.v1.nn.raw_rnn(  
cell, loop_fn, parallel_iterations=None, swap_memory=False,  
scope=None  
)
```

```

time = tf.constant(0, dtype=tf.int32)
(finished, next_input, initial_state, emit_structure, loop_state) =
loop_fn(
time=time, cell_output=None, cell_state=None, loop_state=None)
emit_ta = TensorArray(dynamic_size=True, dtype=initial_state.dtype)
state = initial_state
while not all(finished):
(output, cell_state) = cell(next_input, state)
(next_finished, next_input, next_state, emit, loop_state) =
loop_fn(
time=time + 1, cell_output=output, cell_state=cell_state,
loop_state=loop_state)
# Emit zeros and copy forward state for minibatch entries that are
finished.
state = tf.where(finished, state, next_state)
emit = tf.where(finished, tf.zeros_like(emit_structure), emit)
emit_ta = emit_ta.write(time, emit)
# If any new minibatch entries are marked as finished, mark these.
finished = tf.logical_or(finished, next_finished)
time += 1
return (emit_ta, state, loop_state)

```

```

time = tf.constant(0, dtype=tf.int32)
(finished, next_input, initial_state, emit_structure, loop_state) =
loop_fn(
time=time, cell_output=None, cell_state=None, loop_state=None)
emit_ta = TensorArray(dynamic_size=True, dtype=initial_state.dtype)
state = initial_state
while not all(finished):
(output, cell_state) = cell(next_input, state)
(next_finished, next_input, next_state, emit, loop_state) =
loop_fn(
time=time + 1, cell_output=output, cell_state=cell_state,
loop_state=loop_state)
# Emit zeros and copy forward state for minibatch entries that are
finished.
state = tf.where(finished, state, next_state)
emit = tf.where(finished, tf.zeros_like(emit_structure), emit)
emit_ta = emit_ta.write(time, emit)
# If any new minibatch entries are marked as finished, mark these.
finished = tf.logical_or(finished, next_finished)
time += 1
return (emit_ta, state, loop_state)

```

```

inputs = tf.compat.v1.placeholder(shape=(max_time, batch_size,
input_depth),
dtype=tf.float32)
sequence_length = tf.compat.v1.placeholder(shape=(batch_size,),
dtype=tf.int32)
inputs_ta = tf.TensorArray(dtype=tf.float32, size=max_time)
inputs_ta = inputs_ta.unstack(inputs)
cell = tf.compat.v1.nn.rnn_cell.LSTMCell(num_units)
def loop_fn(time, cell_output, cell_state, loop_state):
    emit_output = cell_output # == None for time == 0
    if cell_output is None: # time == 0
        next_cell_state = cell.zero_state(batch_size, tf.float32)
    else:
        next_cell_state = cell_state
    elements_finished = (time >= sequence_length)
    finished = tf.reduce_all(elements_finished)
    next_input = tf.cond(
        finished,
        lambda: tf.zeros([batch_size, input_depth], dtype=tf.float32),
        lambda: inputs_ta.read(time))
    next_loop_state = None
    return (elements_finished, next_input, next_cell_state,
        emit_output, next_loop_state)
outputs_ta, final_state, _ = raw_rnn(cell, loop_fn)
outputs = outputs_ta.stack()

```

```

inputs = tf.compat.v1.placeholder(shape=(max_time, batch_size,
input_depth),
dtype=tf.float32)
sequence_length = tf.compat.v1.placeholder(shape=(batch_size,),
dtype=tf.int32)
inputs_ta = tf.TensorArray(dtype=tf.float32, size=max_time)
inputs_ta = inputs_ta.unstack(inputs)
cell = tf.compat.v1.nn.rnn_cell.LSTMCell(num_units)
def loop_fn(time, cell_output, cell_state, loop_state):
    emit_output = cell_output # == None for time == 0
    if cell_output is None: # time == 0
        next_cell_state = cell.zero_state(batch_size, tf.float32)
    else:
        next_cell_state = cell_state
    elements_finished = (time >= sequence_length)
    finished = tf.reduce_all(elements_finished)
    next_input = tf.cond(
        finished,
        lambda: tf.zeros([batch_size, input_depth], dtype=tf.float32),
        lambda: inputs_ta.read(time))
    next_loop_state = None
    return (elements_finished, next_input, next_cell_state,
        emit_output, next_loop_state)
outputs_ta, final_state, _ = raw_rnn(cell, loop_fn)
outputs = outputs_ta.stack()

```

Documentation

Creates an RNN specified by RNNCell cell and loop function loop_fn.

This function is a more primitive version of `dynamic_rnn` that provides more direct access to the inputs each iteration. It also provides more control over when to start and finish reading the sequence, and what to emit for the output.

For example, it can be used to implement the dynamic decoder of a seq2seq

model.

Instead of working with Tensor objects, most operations work with TensorArray objects directly.

The operation of raw_rnn, in pseudo-code, is basically the following:

with the additional properties that output and state may be (possibly nested) tuples, as determined by cell.output_size and cell.state_size, and as a result the final state and emit_ta may themselves be tuples.

A simple implementation of dynamic_rnn via raw_rnn looks like this:

Returns

emit_ta: The RNN output TensorArray.

If loop_fn returns a (possibly nested) set of Tensors for emit_output during initialization, (inputs time = 0, cell_output = None, and loop_state = None), then emit_ta will have the same structure, dtypes, and shapes as emit_output instead. If loop_fn returns emit_output = None during this call, the structure of cell.output_size is used:

If cell.output_size is a (possibly nested) tuple of integers or TensorShape objects, then emit_ta will be a tuple having the same structure as cell.output_size, containing TensorArrays whose elements' shapes correspond to the shape data in cell.output_size.

final_state: The final cell state. If cell.state_size is an int, this will be shaped [batch_size, cell.state_size]. If it is a TensorShape, this will be shaped [batch_size] + cell.state_size. If it is a (possibly nested) tuple of ints or TensorShape, this will

be a tuple having the corresponding shapes.

`final_loop_state`: The final loop state as returned by `loop_fn`.

Raises